

Phi_K advanced tutorial

This notebook guides you through the more advanced functionality of the phik package. This notebook will not cover all the underlying theory, but will just attempt to give an overview of all the options that are available. For a theoretical description the user is referred to our paper.

The package offers functionality on three related topics:

1. Phik correlation matrix
2. Significance matrix
3. Outlier significance matrix

```
In [1]: %%capture
# install phik (if not installed yet)
import sys

!"{sys.executable}" -m pip install phik
```

```
In [2]: # import standard packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import itertools

import phik

from phik import resources
from phik.binning import bin_data
from phik.decorators import *
from phik.report import plot_correlation_matrix

%matplotlib inline
```

```
In [3]: # if one changes something in the phik-package one can automatically reload the package or module
%load_ext autoreload
%autoreload 2
```

Load data

A simulated dataset is part of the phik-package. The dataset concerns car insurance data. Load the dataset here:

```
In [4]: data = pd.read_csv( resources.fixture('fake_insurance_data.csv.gz') )
```

```
In [5]: data.head()
```

Out[5]:

	car_color	driver_age	area	mileage	car_size
0	black	26.377219	suburbs	156806.288398	XXL
1	black	58.976840	suburbs	74400.323559	XL
2	multicolor	55.744988	downtown	267856.748015	XXL
3	metallic	57.629139	downtown	259028.249060	XXL
4	green	21.490637	downtown	110712.216080	XL

Specify bin types

The phik-package offers a way to calculate correlations between variables of mixed types. Variable types can be inferred automatically although we recommend to variable types to be specified by the user.

Because interval type variables need to be binned in order to calculate phik and the significance, a list of interval variables is created.

In [6]:

```
data_types = {'severity': 'interval',
              'driver_age': 'interval',
              'satisfaction': 'ordinal',
              'mileage': 'interval',
              'car_size': 'ordinal',
              'car_use': 'ordinal',
              'car_color': 'categorical',
              'area': 'categorical'}

interval_cols = [col for col, v in data_types.items() if v=='interval' and col in data.columns]
interval_cols
# interval_cols is used below
```

Out[6]: ['driver_age', 'mileage']

Phik correlation matrix

Now let's start calculating the correlation phik between pairs of variables.

Note that the original dataset is used as input, the binning of interval variables is done automatically.

In [7]:

```
phik_overview = data.phik_matrix(interval_cols=interval_cols)
phik_overview
```

Out [7]:

	car_color	driver_age	area	mileage	car_size
car_color	1.000000	0.389671	0.590456	0.000000	0.000000
driver_age	0.389671	1.000000	0.105506	0.000000	0.000000
area	0.590456	0.105506	1.000000	0.000000	0.000000
mileage	0.000000	0.000000	0.000000	1.000000	0.768589
car_size	0.000000	0.000000	0.000000	0.768589	1.000000

Specify binning per interval variable

Binning can be set per interval variable individually. One can set the number of bins, or specify a list of bin edges. Note that the measured phik correlation is dependent on the chosen binning. The default binning is uniform between the min and max values of the interval variable.

In [8]:

```
bins = {'mileage':5, 'driver_age':[18,25,35,45,55,65,125]}
phik_overview = data.phik_matrix(interval_cols=interval_cols, bins=bins)
phik_overview
```

Out [8]:

	car_color	driver_age	area	mileage	car_size
car_color	1.000000	0.388350	0.590456	0.000000	0.000000
driver_age	0.388350	1.000000	0.071189	0.000000	0.000000
area	0.590456	0.071189	1.000000	0.000000	0.000000
mileage	0.000000	0.000000	0.000000	1.000000	0.665845
car_size	0.000000	0.000000	0.000000	0.665845	1.000000

Do not apply noise correction

For low statistics samples often a correlation larger than zero is measured when no correlation is actually present in the true underlying distribution. This is not only the case for phik, but also for the pearson correlation and Cramer's phi (see figure 4 in XX). In the phik calculation a noise correction is applied by default, to take into account erroneous correlation values as a result of low statistics. To switch off this noise cancellation (not recommended), do:

In [9]:

```
phik_overview = data.phik_matrix(interval_cols=interval_cols, noise_correction=False)
phik_overview
```

Out [9]:

	car_color	driver_age	area	mileage	car_size
car_color	1.000000	0.407860	0.594172	0.136267	0.096629
driver_age	0.407860	1.000000	0.190390	0.199606	0.121585
area	0.594172	0.190390	1.000000	0.149679	0.067452
mileage	0.136267	0.199606	0.149679	1.000000	0.770836
car_size	0.096629	0.121585	0.067452	0.770836	1.000000

Using a different expectation histogram

By default phik compares the 2d distribution of two (binned) variables with the distribution that assumes no dependency between them. One can also change the expected distribution though. Phi_K is calculated in the same way, but using the other expectation distribution.

```
In [10]: from phik.binning import auto_bin_data
from phik.phik import phik_observed_vs_expected_from_rebinned_df, phik_from_hist2d
from phik.statistics import get_dependent_frequency_estimates
```

```
In [11]: # get observed 2d histogram of two variables
cols = ["mileage", "car_size"]
icols = ["mileage"]
observed = data[cols].hist2d(interval_cols=icols).values
```

```
In [12]: # default phik evaluation from observed distribution
phik_value = phik_from_hist2d(observed)
print (phik_value)
```

0.7685888294891438

```
In [13]: # phik evaluation from an observed and expected distribution
expected = get_dependent_frequency_estimates(observed)
phik_value = phik_from_hist2d(observed=observed, expected=expected)
print (phik_value)
```

0.7685888294891438

```
In [14]: # one can also compare two datasets against each other, and get a full phik matrix that way.
# this needs binned datasets though.
# (the user needs to make sure the binnings of both datasets are identical.)
data_binned, _ = auto_bin_data(data, interval_cols=interval_cols)
```

```
In [15]: # here we are comparing data_binned against itself
phik_matrix = phik_observed_vs_expected_from_rebinned_df(data_binned, data_binned)
```

```
In [16]: # all off-diagonal entries are zero, meaning the all 2d distributions of both datasets are identical.
# (by construction the diagonal is one.)
phik_matrix
```

Out[16]:

	car_color	driver_age	area	mileage	car_size
car_color	1.0	0.0	0.0	0.0	0.0
driver_age	0.0	1.0	0.0	0.0	0.0
area	0.0	0.0	1.0	0.0	0.0
mileage	0.0	0.0	0.0	1.0	0.0
car_size	0.0	0.0	0.0	0.0	1.0

Statistical significance of the correlation

When assessing correlations it is good practise to evaluate both the correlation and the significance of the correlation: a large correlation may be statistically insignificant, and vice versa a small correlation may be very significant. For instance, `scipy.stats.pearsonr` returns both the pearson correlation and the p-value. Similarly, the `phik` package offers functionality to calculate a significance matrix. Significance is defined as:

$$Z = \Phi^{-1}(1-p) \quad \Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^z e^{-t^2/2} dt$$

Several corrections to the 'standard' p-value calculation are taken into account, making the method more robust for low statistics and sparse data cases. The user is referred to our paper for more details.

Due to the corrections, the significance calculation can take a few seconds.

```
In [17]: significance_overview = data.significance_matrix(interval_cols=interval_cols)
significance_overview
```

Out[17]:

	car_color	driver_age	area	mileage	car_size
car_color	85.497902	19.881657	37.613587	-0.507168	-0.450144
driver_age	19.881657	84.365518	1.921501	-0.557628	-0.437694
area	37.613587	1.921501	72.427539	-0.544296	-0.252792
mileage	-0.507168	-0.557628	-0.544296	91.266117	49.262267
car_size	-0.450144	-0.437694	-0.252792	49.262267	69.056588

Specify binning per interval variable

Binning can be set per interval variable individually. One can set the number of bins, or specify a list of bin edges. Note that the measure `phik` correlation is dependent on the chosen binning.

```
In [18]: bins = {'mileage':5, 'driver_age':[18,25,35,45,55,65,125]}
significance_overview = data.significance_matrix(interval_cols=interval_cols, bins=bins)
significance_overview
```

Out[18]:

	car_color	driver_age	area	mileage	car_size
car_color	85.474722	20.547973	37.601726	-0.201770	-0.460109
driver_age	20.547973	83.360009	2.439972	-0.527705	-0.533396
area	37.601726	2.439972	72.433875	-0.267340	-0.294992
mileage	-0.201770	-0.527705	-0.267340	77.770844	47.007380
car_size	-0.460109	-0.533396	-0.294992	47.007380	69.070769

Specify significance method

The recommended method to calculate the significance of the correlation is a hybrid approach, which uses the G-test statistic. The number of degrees of freedom and an analytical, empirical description of the χ^2 distribution are used, based on Monte Carlo simulations. This method works well for both high and low statistics samples.

Other approaches to calculate the significance are implemented:

- asymptotic: fast, but over-estimates the number of degrees of freedom for low statistics samples, leading to erroneous values of the significance

- MC: Many simulated samples are needed to accurately measure significances larger than 3, making this method computationally expensive.

```
In [19]: significance_overview = data.significance_matrix(interval_cols=interval_cols, significance_method='asymptotic')
significance_overview
```

Out[19]:

	car_color	driver_age	area	mileage	car_size
car_color	85.526574	19.681564	37.661844	-0.385023	-0.333340
driver_age	19.681564	84.014654	1.742050	-0.947153	-0.793434
area	37.661844	1.742050	72.440209	-0.465002	-0.123678
mileage	-0.385023	-0.947153	-0.465002	91.301129	49.332305
car_size	-0.333340	-0.793434	-0.123678	49.332305	69.107448

Simulation method

The chi2 of a contingency table is measured using a comparison of the expected frequencies with the true frequencies in a contingency table. The expected frequencies can be simulated in a variety of ways. The following methods are implemented:

- multinomial: Only the total number of records is fixed. (default)
- row_product_multinomial: The row totals fixed in the sampling.
- col_product_multinomial: The column totals fixed in the sampling.
- hypergeometric: Both the row or column totals are fixed in the sampling. (Note that this type of sampling is only available when row and column totals are integers, which is usually the case.)

```
In [20]: # --- Warning, can be slow
#         turned off here by default for unit testing purposes

#significance_overview = data.significance_matrix(interval_cols=interval_cols, simulation_method='hypergeometric')
#significance_overview
```

Expected frequencies

```
In [21]: from phik.simulation import sim_2d_data_patefield, sim_2d_product_multinomial, sim_2d_data
```

```
In [22]: inputdata = data[['driver_age', 'area']].hist2d(interval_cols=['driver_age'])
inputdata
```


Out[22]:

	area	country_side	downtown	hills	suburbs	unpaved_roads
driver_age						
1		11.0	86.0	123.0	147.0	21.0
2		9.0	77.0	137.0	125.0	31.0
3		7.0	102.0	131.0	130.0	18.0
4		17.0	83.0	130.0	95.0	14.0
5		13.0	68.0	120.0	72.0	8.0
6		7.0	30.0	51.0	47.0	9.0
7		1.0	11.0	23.0	14.0	7.0
8		0.0	4.0	7.0	8.0	2.0
9		0.0	0.0	1.0	1.0	0.0
10		0.0	1.0	1.0	0.0	0.0

Multinomial

```
In [23]: simdata = sim_2d_data(inputdata.values)
print('data total:', inputdata.sum().sum())
print('sim total:', simdata.sum().sum())
print('data row totals:', inputdata.sum(axis=0).values)
print('sim row totals:', simdata.sum(axis=0))
print('data column totals:', inputdata.sum(axis=1).values)
print('sim column totals:', simdata.sum(axis=1))
```

data total: 2000.0
sim total: 2000
data row totals: [65. 462. 724. 639. 110.]
sim row totals: [67 459 734 636 104]
data column totals: [388. 379. 388. 339. 281. 144. 56. 21. 2. 2.]
sim column totals: [377 393 398 332 294 135 51 16 1 3]

product multinomial

```
In [24]: simdata = sim_2d_product_multinomial(inputdata.values, axis=0)
print('data total:', inputdata.sum().sum())
print('sim total:', simdata.sum().sum())
print('data row totals:', inputdata.sum(axis=0).astype(int).values)
print('sim row totals:', simdata.sum(axis=0).astype(int))
print('data column totals:', inputdata.sum(axis=1).astype(int).values)
print('sim column totals:', simdata.sum(axis=1).astype(int))
```

data total: 2000.0
sim total: 2000
data row totals: [65 462 724 639 110]
sim row totals: [65 462 724 639 110]
data column totals: [388 379 388 339 281 144 56 21 2 2]
sim column totals: [374 372 402 336 292 144 60 18 1 1]

hypergeometric ("patefield")

```
In [25]: # patefiled simulation needs compiled c++ code.
# only run this if the python binding to the (compiled) patefiled simulation function is found.
try:
    from phik.simcore import _sim_2d_data_patefield
    CPP_SUPPORT = True
except ImportError:
    CPP_SUPPORT = False

if CPP_SUPPORT:
    simdata = sim_2d_data_patefield(inputdata.values)
    print('data total:', inputdata.sum().sum())
    print('sim total:', simdata.sum().sum())
    print('data row totals:', inputdata.sum(axis=0).astype(int).values)
    print('sim row totals:', simdata.sum(axis=0))
    print('data column totals:', inputdata.sum(axis=1).astype(int).values)
    print('sim column totals:', simdata.sum(axis=1))
```

```
data total: 2000.0
sim  total: 2000
data row totals: [ 65 462 724 639 110]
sim  row totals: [ 65 462 724 639 110]
data column totals: [388 379 388 339 281 144 56 21 2 2]
sim  column totals: [388 379 388 339 281 144 56 21 2 2]
```

Outlier significance

The normal pearson correlation between two interval variables is easy to interpret. However, the phik correlation between two variables of mixed type is not always easy to interpret, especially when it concerns categorical variables. Therefore, functionality is provided to detect "outliers": excesses and deficits over the expected frequencies in the contingency table of two variables.

Example 1: mileage versus car_size

For the categorical variable pair mileage - car_size we measured:

$$\phi_k = 0.77 \text{ , } \quad \quad \mathrm{significance} = 46.3\%$$

Let's use the outlier significance functionality to gain a better understanding of this significance correlation between mileage and car size.

[illegible]

Out[27]:

car_size	L	M	S	XL	XS	XXL
53.5_30047.0	6.882155	21.483476	18.076204	-10.367152	10.820863	-22.423985
30047.0_60040.5	20.034528	-0.251737	-3.408409	2.534277	-1.973628	-16.087822
60040.5_90033.9	1.627610	-3.043497	-2.265809	10.215936	-1.246784	-8.318068
90033.9_120027.4	-3.711579	-3.827278	-2.885475	12.999048	-1.638288	-7.185622
120027.4_150020.9	-7.665861	-6.173001	-4.746762	9.629145	-2.841508	-0.504521
150020.9_180014.4	-7.533189	-6.063786	-4.660049	1.559370	-2.785049	6.765549
180014.4_210007.8	-5.541940	-4.425929	-3.360023	-4.802787	-1.942469	10.520540
210007.8_240001.3	-3.496905	-2.745103	-2.030802	-5.850529	-1.100873	8.723925
240001.3_269994.8	-5.275976	-4.207164	-3.186534	-8.616464	-1.830944	13.303101
269994.8_299988.2	-8.014016	-6.458253	-4.973240	-12.868389	-2.989055	20.992824

Specify binning per interval variable

Binning can be set per interval variable individually. One can set the number of bins, or specify a list of bin edges.

Note: in case a bin is created without any records this bin will be automatically dropped in the phik and (outlier) significance calculations. However, in the outlier significance calculation this will currently lead to an error as the number of provided bin edges does not match the number of bins anymore.

In [28]:

```
bins = [0, 1E2, 1E3, 1E4, 1E5, 1E6]
outlier_signifs, binning_dict = data[[c0, c1]].outlier_significance_matrix(interval_cols=tmp_interval_cols,
                                                                           bins=bins, retbins=True)
outlier_signifs
```

Out[28]:

car_size	L	M	S	XL	XS	XXL
0.0_100.0	-0.223635	-0.153005	-0.096640	-0.504167	2.150837	-1.337308
100.0_1000.0	-0.742899	-0.533211	2.164954	-1.469996	5.704340	-3.272689
1000.0_10000.0	-3.489668	3.499856	18.061724	-6.831062	11.617394	-13.063085
10000.0_100000.0	25.086723	15.956527	-0.251877	5.162309	-3.896807	-24.429042
100000.0_1000000.0	-20.527411	-17.223164	-13.626621	-2.140870	-8.688844	44.933133

Specify binning per interval variable -- dealing with underflow and overflow

When specifying custom bins as situation can occur when the minimal (maximum) value in the data is smaller (larger) than the minimum (maximum) bin edge. Data points outside the specified range will be collected in the underflow (UF) and overflow (OF) bins. One can choose how to deal with these under/overflow bins, by setting the `drop_underflow` and `drop_overflow` variables.

Note that the `drop_underflow` and `drop_overflow` options are also available for the calculation of the `phik` matrix and the significance matrix.

In [29]:

[illegible]

```
outlier_signifs = drop_underflow=False,  
drop_overflow=False)
```

car_size	L	M	S	XL	XS	XXL
100.0_1000.0	-0.742899	-0.533211	2.164954	-1.469996	5.704340	-3.272689
1000.0_10000.0	-3.489668	3.499856	18.061724	-6.831062	11.617394	-13.063085
10000.0_100000.0	25.086723	15.956527	-0.251877	5.162309	-3.896807	-24.429042
OF	-20.527411	-17.223164	-13.626621	-2.140870	-8.688844	44.933133
UF	-0.223635	-0.153005	-0.096640	-0.504167	2.150837	-1.337308

Dealing with NaN's in the data

Let's add some missing values to our data

```
data.loc[np.random.choice(range(len(data)), size=10), 'car_size'] = np.nan
data.loc[np.random.choice(range(len(data)), size=10), 'mileage'] = np.nan
```

Sometimes there can be information in the missing values and in which case you might want to consider the NaN values as a separate category. This can be achieved by setting the `dropna` argument to `False`.

[illegible]

car_size	L	M	NaN	S	XL	XS	XXL
100.0_1000.0	-0.739557	-0.533211	-0.053620	2.164954	-1.464646	5.704340	-3.254118
1000.0_10000.0	-3.475579	3.499856	-0.580372	18.061724	-6.811959	11.617394	-13.000691
10000.0_100000.0	25.043773	15.990553	-1.085685	-0.230920	5.041389	-3.886172	-24.277670
NaN	-0.948254	-0.693311	-0.073439	-0.465651	0.577126	-0.211155	0.610910
OF	-20.381090	-17.137612	1.129231	-13.554215	-2.122236	-8.639494	43.494996
UF	-0.222469	-0.153005	-0.013130	-0.096640	-0.501935	2.150837	-1.328194

Here OF and UF are the underflow and overflow bin of car_size, respectively.

To just ignore records with missing values set `dropna` to `True` (default).

[illegible]

outlier_signifs

dropna=True)

Out[32]:

car_size	L	M	S	XL	XS	XXL
100.0_1000.0	-0.745805	-0.537881	2.157106	-1.468203	5.695755	-3.265980
1000.0_10000.0	-3.502461	3.468653	18.028651	-6.825512	11.594866	-13.041086
10000.0_100000.0	24.987515	15.937493	-0.271439	5.036675	-3.906908	-24.292311
OF	-20.433225	-17.189419	-13.599274	-2.017897	-8.670563	44.692414
UF	-0.224643	-0.154513	-0.097599	-0.503408	2.146765	-1.334000

Note that the dropna option is also available for the calculation of the phik matrix and the significance matrix.